A Single-Level MILP Formulation for Electric Vehicle Charger Location and Sizing

Technical Report

Contents

1 Overview

This report documents the mathematical formulation solved by the Julia/JuMP code in `charger_model.jl`. The model decides *where* to open electric-vehicle (EV) charging stations, *how many servers (chargers) to install* at each opened station, and *how to route demand* from origin nodes to stations—subject to a budget constraint—while maximising the total load served and penalising queueing waiting time.

The key modelling challenge is that waiting time at each station is a *non-linear, convex* function of the arrival load (derived from an $M/M/s$ queue). The code resolves this via a **piecewise-linear (PWL) epigraph approximation** and embeds the lower-level user-assignment problem directly into a **single-level MILP** through KKT / strong-duality conditions. The resulting model is solved with Gurobi.

2 Sets and Indices

Symbol	Description
$I = \{1, \ldots, n\}$	Origin (demand) nodes
$J = \{1, \ldots, n\}$	Candidate station locations (same ground set as I)
$S = \{1, \ldots, s_{\max}\}$	Discrete server-count options per location
$K = \{1, \ldots, N_{\text{bp}}\}$	Breakpoint indices for the PWL waiting approximation

Table 1: Sets and indices.

In the current configuration: $n = 30$ (clustered nodes for Milano), $s_{\max} = 6$, $N_{\text{bp}} = 100$.

3 Parameters

3.1 Demand Scaling

Raw demand $\bar{d}_i$ (assigned population at each clustered node) is scaled by a factor σ to bring average demand close to 1:

$$\sigma = \frac{1}{\bar{\bar{d}}} \quad \text{(when } \texttt{auto\_normalize\_demand} \text{ is set)}, \qquad d_i = \sigma \cdot \bar{d}_i.$$

3.2 PWL Waiting Approximation

For each (j, s) pair, the breakpoints are a uniform grid on $[0, \min(\lambda^{\max}_{\text{global}}, \text{cap}_{js})]$:

$$\hat{\lambda}_{jsk} = (k-1)\,\frac{\min(\lambda^{\max}, \text{cap}_{js})}{N_{\text{bp}} - 1}, \quad k = 1, \ldots, N_{\text{bp}}.$$

Symbol	Type	Description
d_i	$\mathbb{R}_+$	Effective (scaled) demand / arrival rate at origin i
t_{ij}	$\mathbb{R}_+$	Travel-time proxy from origin i to location j
c_j	$\mathbb{R}_+$	Cost per server installed at location j (default 1.0)
γ	$\mathbb{R}_+$	Total server budget (default 8)
μ_j	$\mathbb{R}_+$	Service rate per server at location j
$\bar{q}$	$\mathbb{R}_+$	Capacity per server: $\bar{q} = \mu_j - \varepsilon$ (default 4.0)
cap_{js}	$\mathbb{R}_+$	Max stable load: $\text{cap}_{js} = s \cdot \bar{q}$
r_i	$\mathbb{R}_+$	No-option cost for origin i (default 30.0)
α	$\mathbb{R}_+$	Waiting-time penalty weight (default 1.0)
$\hat{\lambda}_{jsk}$	$\mathbb{R}$	k-th breakpoint for station j, server count s
w_{jsk}	$\mathbb{R}_+$	Expected waiting time at breakpoint k: $W_{M/M/s}(s, \mu_j, \hat{\lambda}_{jsk})$
F_{jsk}	$\mathbb{R}_+$	Cumulative integral of w up to breakpoint k (trapezoid rule)

Table 2: Parameters.

The waiting time at each breakpoint uses the $M/M/s$ Erlang-C formula:

$$w_{jsk} = W_{M/M/s}(s, \mu_j, \hat{\lambda}_{jsk}) = \frac{C(s, \hat{\lambda}_{jsk}/\mu_j)}{s\mu_j - \hat{\lambda}_{jsk}},$$

where

$$C(s,a) = \frac{a^s/(s!\,(1-\rho))}{\sum_{k=0}^{s-1} a^k/k! \ + \ a^s/(s!\,(1-\rho))}, \quad \rho = \frac{a}{s} = \frac{\lambda}{s\mu}.$$

The cumulative integral is approximated by the trapezoid rule:

$$F_{js1} = 0, \qquad F_{jsk} = F_{js,k-1} + \tfrac{1}{2}(w_{js,k-1} + w_{jsk})(\hat{\lambda}_{jsk} - \hat{\lambda}_{js,k-1}), \quad k \geq 2.$$

4 Decision Variables

5 Full Single-Level MILP Formulation

5.1 Objective

$$\boxed{\max_{x,\xi,y,y^0,\lambda,\phi,\boldsymbol{\pi}} \sum_{j \in J} \lambda_j} \tag{OBJ}$$

The objective maximises total load routed to open stations (equivalently, minimises unserved demand).

Variable	Domain	Interpretation
x_{js}	$\{0,1\}$	1 if location j opens with exactly s servers
ξ_j	$\{0,1\}$	1 if location j is open ($\xi_j = \sum_s x_{js}$)
y_{ij}	$\mathbb{R}_+$	Demand flow from origin i assigned to station j
y_i^0	$\mathbb{R}_+$	Demand at origin i that goes unserved (no-option)
λ_j	$\mathbb{R}_+$	Total arrival rate at station j
ϕ_j	$\mathbb{R}_+$	Epigraph variable for waiting cost at station j
Follower dual variables (KKT embedding)		
π_i^d	$\mathbb{R}$	Dual for demand satisfaction at origin i
π_j^λ	$\mathbb{R}$	Dual for load definition at station j
π_{ij}^x	$\mathbb{R}_+$	Dual for access restriction at (i,j)
π_{js}^s	$\mathbb{R}_+$	Dual for stability constraint at (j,s)
π_{jsk}^w	$\mathbb{R}_+$	Dual for PWL waiting constraint at (j,s,k)
Π_{ij}	$\mathbb{R}_+$	Linearisation of $x_{js} \cdot \pi_{ij}^x$

Table 3: Decision variables.

5.2 Leader Constraints (Facility Design)

Budget:

$$\sum_{j\in J}\sum_{s\in S} c_j \cdot s \cdot x_{js} \;\le\; \gamma \tag{C1}$$

At most one server level per location:

$$\sum_{s\in S} x_{js} \;\le\; 1 \qquad \forall j \in J \tag{C2}$$

Open-indicator consistency:

$$\xi_j \;=\; \sum_{s\in S} x_{js} \qquad \forall j \in J \tag{C3}$$

5.3 Follower Primal Constraints (Demand Assignment)

Demand satisfaction:

$$\sum_{j\in J} y_{ij} + y_i^0 \;=\; d_i \qquad \forall i \in I \tag{C4}$$

Access (only open stations can receive flow):

$$y_{ij} \;\le\; d_i \cdot \xi_j \qquad \forall i \in I,\, j \in J \tag{C5}$$

Load definition:

$$\lambda_j \;=\; \sum_{i \in I} y_{ij} \qquad \forall j \in J \tag{C6}$$

Stability (load must not exceed capacity):

$$\lambda_j \;\leq\; \sum_{s \in S} \mathrm{cap}_{js} \cdot x_{js} \qquad \forall j \in J \tag{C7}$$

5.4 Waiting Cost Epigraph (PWL Activation)

When station j is open with server count s, the waiting cost is lower-bounded by the PWL outer-approximation of the integral of $W_{M/M/s}$:

$$x_{js} = 1 \;\Rightarrow\; \phi_j \;\geq\; F_{jsk} + w_{jsk}\,(\lambda_j - \hat{\lambda}_{jsk}) \qquad \forall j \in J,\, s \in S,\, k \in K \tag{C8}$$

When station j is closed:

$$\xi_j = 0 \;\Rightarrow\; \phi_j = 0 \qquad \forall j \in J \tag{C9}$$

Constraints (C8)–(C9) are encoded as indicator constraints in JuMP/Gurobi, so no big-M coefficient is required.

5.5 Follower Dual Feasibility (KKT Conditions)

For the lower-level assignment problem (with fixed x, ξ) to be optimal, its dual must be feasible:

Dual of $y_{ij} \geq 0$:

$$t_{ij} + \pi^d_i + \pi^x_{ij} - \pi^\lambda_j \;\geq\; 0 \qquad \forall i \in I,\, j \in J \tag{C10}$$

Dual of $y^0_i \geq 0$:

$$r_i + \pi^d_i \;\geq\; 0 \qquad \forall i \in I \tag{C11}$$

Dual of $\lambda_j \geq 0$:

$$\pi^\lambda_j + \sum_{s \in S} \pi^s_{js} + \sum_{s \in S} \sum_{k \in K} w_{jsk}\,\pi^w_{jsk} \;\geq\; 0 \qquad \forall j \in J \tag{C12}$$

Dual of $\phi_j \geq 0$:

$$\alpha - \sum_{s \in S} \sum_{k \in K} \pi^w_{jsk} \;\geq\; 0 \qquad \forall j \in J \tag{C13}$$

5.6 Strong Duality (Lower-Level Optimality)

The strong-duality equality replaces the complementary-slackness conditions and collapses the bilevel structure into a single level:

$$\begin{aligned}
&\sum_{i,j} t_{ij}\, y_{ij} + \sum_{i} r_i\, y_i^0 + \alpha \sum_{j} \phi_j \\
= \; &- \sum_{i} d_i\, \pi_i^d - \sum_{i,j} d_i\, \Pi_{ij} - \sum_{j,s} \mathrm{cap}_{js}\, \pi_{js}^s \\
&+ \sum_{j,s,k} (F_{jsk} - w_{jsk}\, \hat{\lambda}_{jsk})\, \pi_{jsk}^w
\end{aligned} \tag{C14}$$

The right-hand side is the Lagrangian dual value of the lower-level problem (with Π_{ij} linearising bilinear terms; see Section 5.7).

5.7 Bilinear Linearisation

The product $x_{js} \cdot \pi_{ij}^x$ appears in the dual objective. It is linearised with the auxiliary variable Π_{ij} using indicator constraints:

$$x_{js} = 1 \;\Rightarrow\; \Pi_{ij} = \pi_{ij}^x \qquad \forall i \in I,\, j \in J,\, s \in S \tag{C15}$$

$$\xi_j = 0 \;\Rightarrow\; \Pi_{ij} = 0 \qquad \forall i \in I,\, j \in J \tag{C16}$$

6 Complete Formulation (Compact Form)

$$
\begin{aligned}
\max \quad & \sum_{j \in J} \lambda_j && \text{(OBJ)} \\
\text{s.t.} \quad & \sum_{j,s} c_j s\, x_{js} \leq \gamma && \text{(C1)} \\
& \sum_{s} x_{js} \leq 1 \quad \forall j && \text{(C2)} \\
& \xi_j = \sum_{s} x_{js} \quad \forall j && \text{(C3)} \\
& \sum_{j} y_{ij} + y_i^0 = d_i \quad \forall i && \text{(C4)} \\
& y_{ij} \leq d_i\, \xi_j \quad \forall i, j && \text{(C5)} \\
& \lambda_j = \sum_{i} y_{ij} \quad \forall j && \text{(C6)} \\
& \lambda_j \leq \sum_{s} \mathrm{cap}_{js}\, x_{js} \quad \forall j && \text{(C7)} \\
& x_{js} = 1 \Rightarrow \phi_j \geq F_{jsk} + w_{jsk}(\lambda_j - \hat{\lambda}_{jsk}) \quad \forall j, s, k && \text{(C8)} \\
& \xi_j = 0 \Rightarrow \phi_j = 0 \quad \forall j && \text{(C9)} \\
& t_{ij} + \pi_i^d + \pi_{ij}^x - \pi_j^\lambda \geq 0 \quad \forall i, j && \text{(C10)} \\
& r_i + \pi_i^d \geq 0 \quad \forall i && \text{(C11)} \\
& \pi_j^\lambda + \sum_{s} \pi_{js}^s + \sum_{s,k} w_{jsk} \pi_{jsk}^w \geq 0 \quad \forall j && \text{(C12)} \\
& \alpha - \sum_{s,k} \pi_{jsk}^w \geq 0 \quad \forall j && \text{(C13)} \\
& \text{(C14 – strong duality)} \\
& x_{js} = 1 \Rightarrow \Pi_{ij} = \pi_{ij}^x \quad \forall i, j, s && \text{(C15)} \\
& \xi_j = 0 \Rightarrow \Pi_{ij} = 0 \quad \forall i, j && \text{(C16)} \\
& x_{js} \in \{0,1\},\ \xi_j \in \{0,1\} \quad \forall j, s \\
& y_{ij}, y_i^0, \lambda_j, \phi_j \geq 0 \quad \forall i, j \\
& \pi_i^d \in \mathbb{R},\ \pi_j^\lambda \in \mathbb{R} \quad \forall i, j \\
& \pi_{ij}^x, \pi_{js}^s, \pi_{jsk}^w, \Pi_{ij} \geq 0 \quad \forall i, j, s, k
\end{aligned}
$$

7 Default Configuration Parameters

The following table lists every tunable parameter with the value used in the code's `ARGS_OVERRIDE` block.

Parameter	Value	Role
`city`	`Milano`	Instance city / data source
`k`	30	Number of demand clusters ($n = 30$)
`timelimit`	30.0 s	Gurobi wall-clock limit
`alpha_wait` (α)	1.0	Waiting-cost penalty weight
`breakpoints` (N_{bp})	100	PWL grid resolution
`servers_max` ($s_{\max}$)	6	Maximum servers per location
`charger_budget` (γ)	8	Total server budget
`station_capacity_per_server` ($\bar{q}$)	4.0	Service capacity per server
μ_j	$\bar{q} + \varepsilon = 4.0001$	Service rate (internal, ensures stability)
`lambda_scale`	1.0	Global demand scaling multiplier
`demand_scale`	1.0	Additional per-node demand multiplier
`auto_normalize_demand`	true	Normalize so $\bar{d} \approx 1$
`noopt_max_range` (r_i)	30.0	No-option penalty (uniform across i)

Table 4: Configuration parameters and their default/override values.

8 Lower-Level Problem (Follower Subproblem)

Given fixed leader decisions (x, ξ), the users solve a minimum-cost flow / assignment problem:

$$\min_{y, y^0, \lambda, \phi} \quad \sum_{i,j} t_{ij}\, y_{ij} + \sum_i r_i\, y_i^0 + \alpha \sum_j \phi_j \tag{1}$$

$$\text{s.t.} \quad \sum_j y_{ij} + y_i^0 = d_i \quad \forall i \tag{2}$$

$$y_{ij} \le d_i\, \xi_j \quad \forall i, j \tag{3}$$

$$\lambda_j = \sum_i y_{ij} \quad \forall j \tag{4}$$

$$\lambda_j \le \sum_s \text{cap}_{js}\, x_{js} \quad \forall j \tag{5}$$

$$x_{js} = 1 \Rightarrow \phi_j \ge F_{jsk} + w_{jsk}(\lambda_j - \hat{\lambda}_{jsk}) \quad \forall j, s, k \tag{6}$$

$$y_{ij}, y_i^0, \lambda_j, \phi_j \ge 0. \tag{7}$$

Because ϕ_j enters linearly and $w_{jsk} \ge 0$, this lower-level LP is convex (in fact linear after the PWL activation is conditioned on x). Its Lagrangian dual is the right-hand side of strong-duality constraint (C14).

9 Solution Structure and Output

After Gurobi returns:

1. **Opened locations:** $\mathcal{O} = \{(j, s) : x^*_{js} \geq 0.5\}$.
2. **Objective value:** $\sum_j \lambda^*_j$ — total arrival rate served.
3. **No-option flow:** $\sum_i y^{0*}_i$ — demand that reaches no station.
4. **Total installed servers:** $\sum_{j,s} s\, x^*_{js}$.
5. **Assignment plot:** generated via `plot_charger_solution`, using t, d, and the recovered y^* matrix.

10 Replication Guide

10.1 Software Requirements

Component	Version tested	Notes
Julia	≥ 1.9	https://julialang.org
JuMP	≥ 1.15	`Pkg.add("JuMP")`
Gurobi	≥ 10	Requires valid academic/commercial licence
Gurobi.jl	≥ 1.2	Julia wrapper for Gurobi
Statistics	stdlib	Included with Julia

Table 5: Software dependencies.

10.2 File Structure

```
project/
+-- charger_model.jl          # Main script (documented here)
+-- network_design.jl         # Defines: generate_full_instance,
|                             #   print_instance, plot_charger_solution,
|                             #   GurobiFactory
+-- src/
    +-- charger_CLI.jl        # Defines: make_charger_arg_parser,
                              #   parse_charger_commandline
```

10.3 Step-by-Step Replication

1. Install Julia and Gurobi; configure the `GUROBI_HOME` and `GRB_LICENSE_FILE` environment variables.

2. Activate the project environment and install Julia packages:

   ```
   ] activate .
   ] add JuMP Gurobi Statistics
   ```

3. Ensure `network_design.jl` and `src/charger_CLI.jl` are present and expose the four required symbols listed above.

4. Run the main script from the Julia REPL or command line:

   ```
   julia charger_model.jl
   ```

5. To override any parameter, edit the `ARGS_OVERRIDE` vector at the top of the script. For example, to change the city and budget:

   ```
   const ARGS_OVERRIDE = [
       "--city", "Roma",
       "--charger_budget", "12",
       ...
   ]
   ```

10.4 Reproducing the Exact Run Reported Here

Use the following `ARGS_OVERRIDE` block verbatim:

```
const ARGS_OVERRIDE = [
    "--city",                          "Milano",
    "--k",                             "30",
    "--timelimit",                     "30.0",
    "--alpha_wait",                    "1.0",
    "--breakpoints",                   "100",
    "--servers_max",                   "6",
    "--charger_budget",                "8",
    "--station_capacity_per_server", "4.0",
    "--lambda_scale",                  "1.0",
    "--demand_scale",                  "1.0",
    "--auto_normalize_demand",
    "--noopt_max_range",               "30.0",
]
```

Gurobi is invoked single-threaded (`Threads => 1`) with a 30-second wall-clock limit. Results may vary across solver versions due to numerical tolerances and branching heuristics.

11 Complexity and Model Size

With n nodes, $s_{\max}$ server options, and N_{bp} breakpoints:

The dominant cost is the $n \cdot s_{\max} \cdot N_{\text{bp}}$ indicator constraints for the PWL waiting epigraph. Reducing `breakpoints` trades approximation accuracy for solve speed.

Component	**Count (general)**	**Count ($n = 30, s_{\max} = 6, N_{\text{bp}} = 100$)**
Binary variables (x_{js})	$n \cdot s_{\max}$	180
Binary variables (ξ_j)	n	30
Continuous variables	$O(n^2 + n s_{\max} N_{\text{bp}})$	$\approx$ 19,000
Indicator constraints (C8)	$n \cdot s_{\max} \cdot N_{\text{bp}}$	18,000
Dual feasibility constraints	$O(n^2 + n s_{\max})$	$\approx$ 1,080
Strong duality	1	1

Table 6: Model size at the default configuration.